```
  let data = await s3.getObject({                                    ①
    'Bucket': process.env.ImageBucket,
    'Key': image.rawS3Key
  }).promise();
  await fs.writeFile(rawFile, data.Body,
➥ {'encoding': null});                                              ②
  let lenna = await Jimp.read(rawFile);                              ③
  await lenna.sepia().write(processedFile);                          ④
  await fs.unlink(rawFile);                                          ⑤
  let buf = await fs.readFile(processedFile,
➥ {'encoding': null});                                              ⑥
  await s3.putObject({                                               ⑦
    'Bucket': process.env.ImageBucket,
    'Key': processedS3Key,
    'ACL': 'public-read',
    'Body': buf,
    'ContentType': 'image/png'
  }).promise();
  await fs.unlink(processedFile);                                    ⑧
  return processedS3Key;
}

async function processed(image) {
  let processedS3Key = await processImage(image);
  await db.updateItem({                                              ⑨
    'Key': {
      'id': {
        'S': image.id
      }
    },
    'UpdateExpression':
      'SET #s=:newState, version=:newVersion,
➥ processedS3Key=:processedS3Key',                                  ⑩
    'ConditionExpression':
      'attribute_exists(id) AND version=:oldVersion
➥ AND #s IN (:stateUploaded, :stateProcessed)',
    'ExpressionAttributeNames': {
      '#s': 'state'
    },
    'ExpressionAttributeValues': {
      ':newState': {'S': 'processed'},
      ':oldVersion': {'N': image.version.toString()},
      ':newVersion': {'N': (image.version + 1).toString()},
      ':processedS3Key': {'S': processedS3Key},
      ':stateUploaded': {'S': 'uploaded'},
      ':stateProcessed': {'S': 'processed'}
    },
    'ReturnValues': 'ALL_NEW',
```